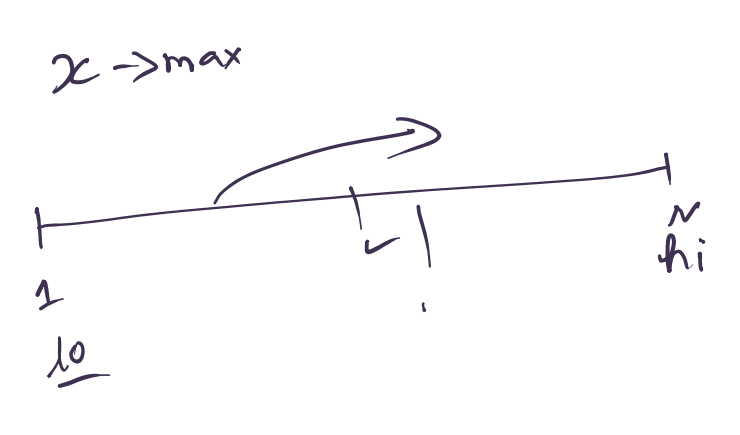
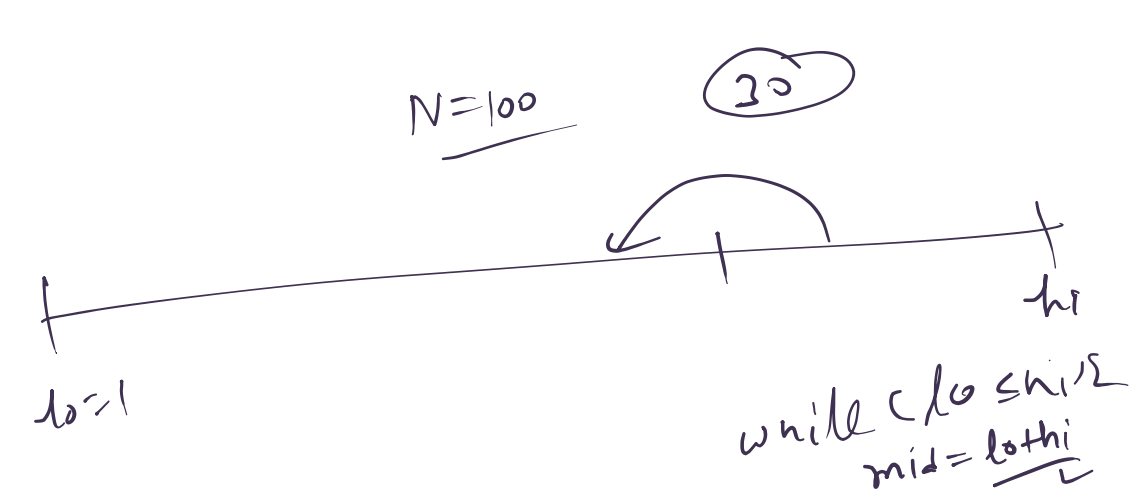
# Lec-7

15/06/26 10:20 AM

$x^K \leq N$

while (lo ≤ hi) {
mid = (lo+hi)/2
if (mid$^K$ ≤ n) {
ans = mid
lo = mid+1
}
else
hi = mid-1
}





while (lo ≤ hi) {
mid = (lo+hi)/2
if (isPossible(mid)) {
ans = mid
hi = mid
}
}

Farmer John has built a new long barn, with N (2 ≤ N ≤ 100,000) stalls. The stalls are located along a straight line at positions x1 ... xN (0 ≤ xi ≤ 1,000,000,000).

His C (2 ≤ C ≤ N) cows don't like this barn layout and become aggressive towards each other once put into a stall. To prevent the cows from hurting each other, FJ wants to assign the cows to the stalls, such that the minimum distance between any two of them is as large as possible. What is the largest minimum distance?

10

5

## Input

*t* – the number of test cases, then *t* test cases follows.

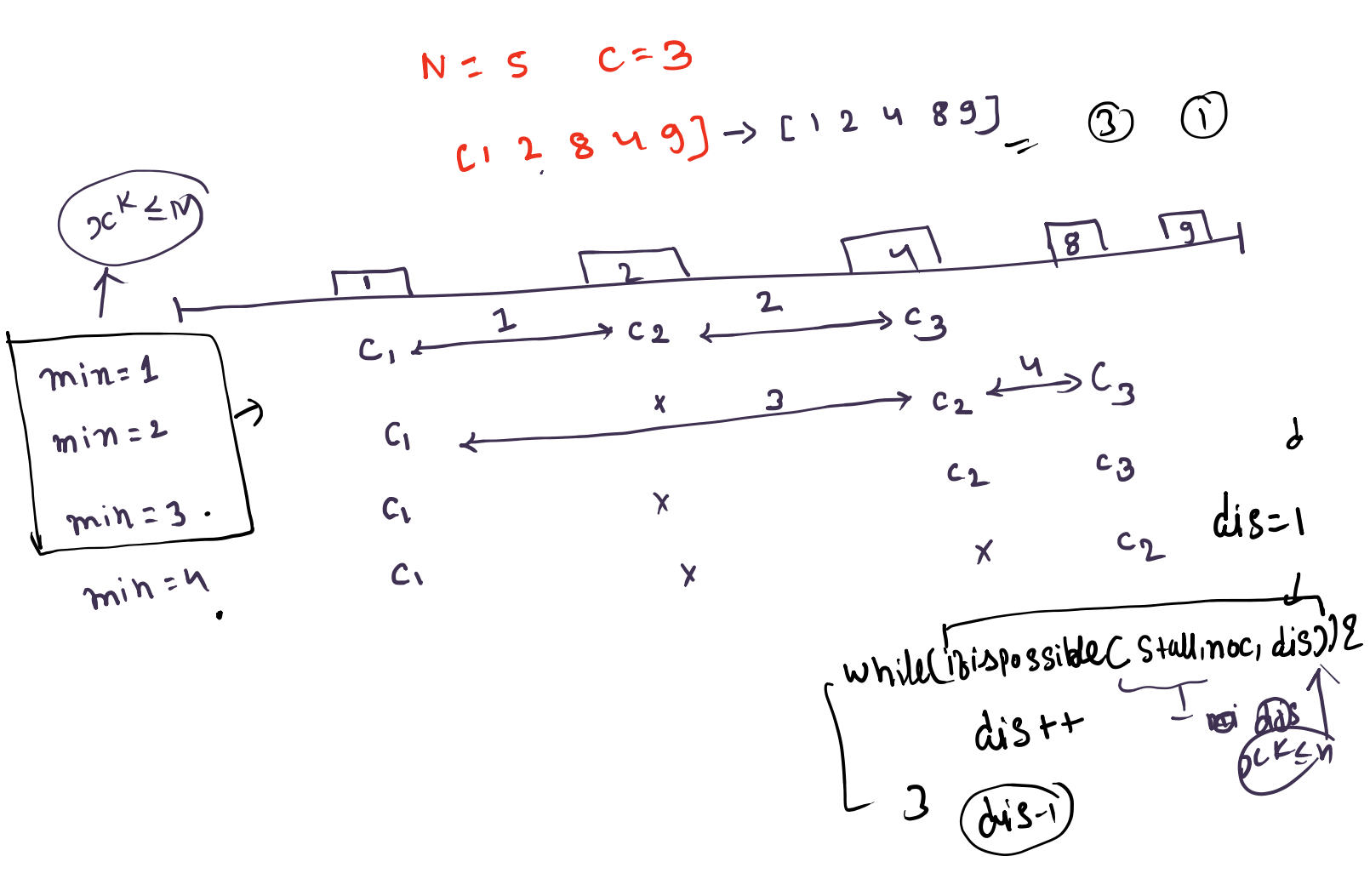
- Line 1: Two space-separated integers: N and C
- Lines 2..N+1: Line i+1 contains an integer stall location, xi

## Output

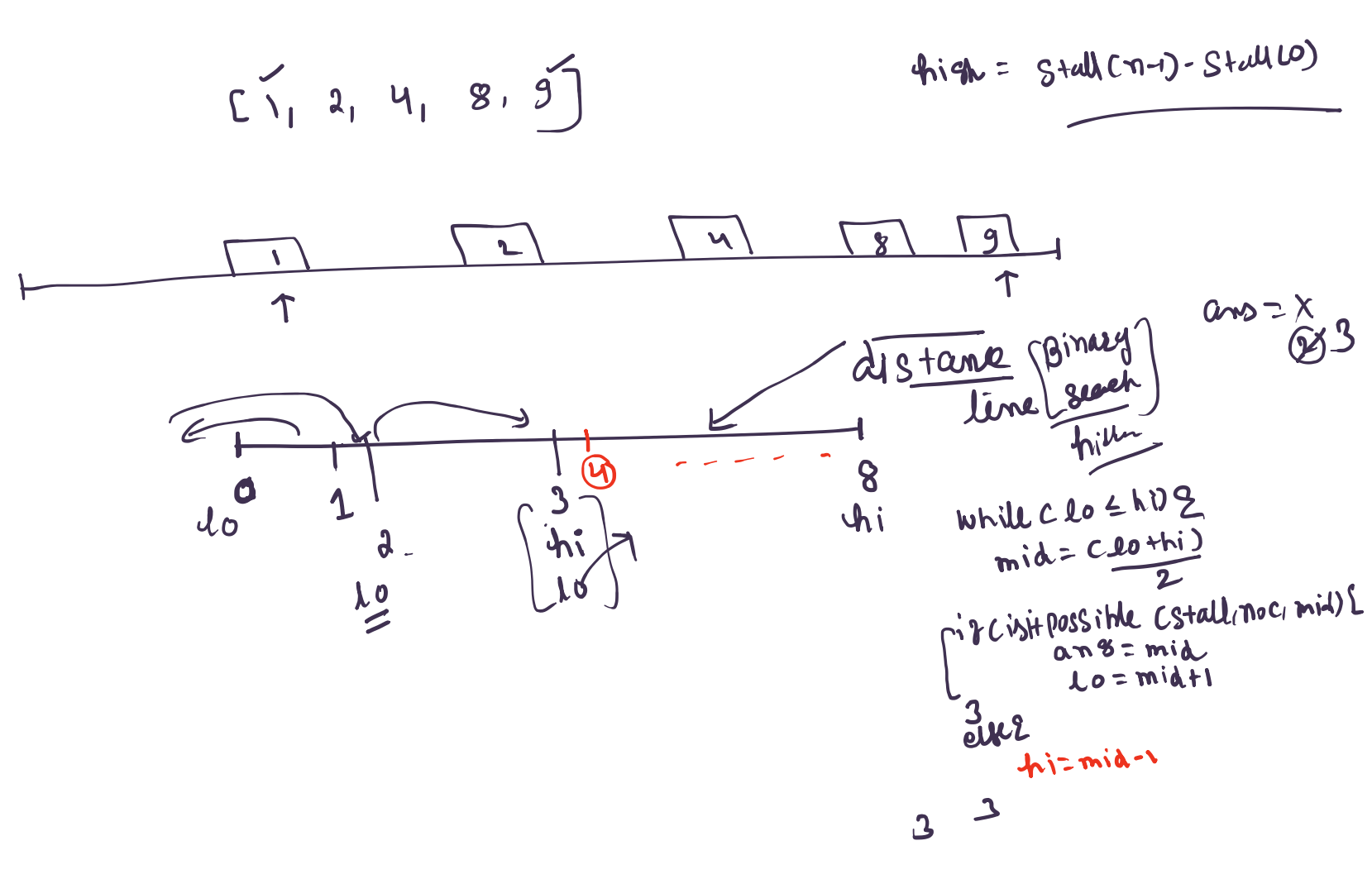
For each test case output one integer: the largest minimum distance.

N = 5  C = 3

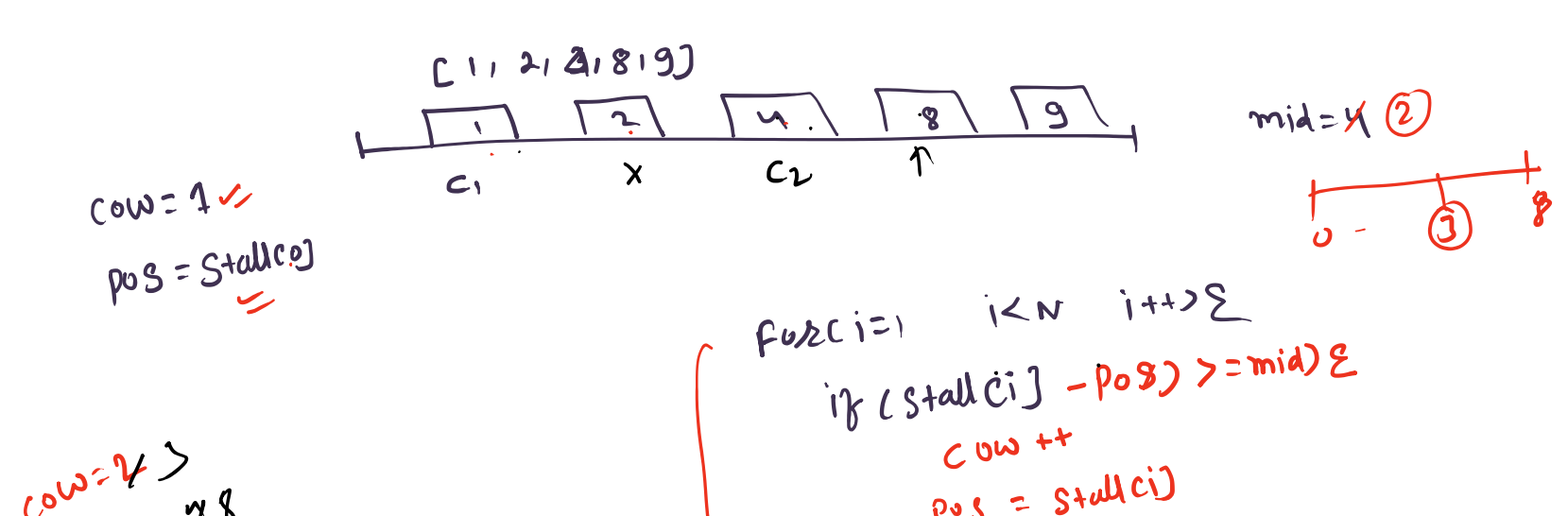
[1 2 8 4 9] → [1 2 4 8 9]  ③ ①



while (isPossible(Stall, noc, dis)) {
dis++
}
(dis-1)
ans
$x^K \leq N$



while (lo ≤ hi) {
mid = (lo+hi)/2
if (isPossible(stall, noc, mid)) {
ans = mid
lo = mid+1
}
else {
hi = mid-1
}
}



for (i=1; i<N; i++) {
if (stall[i] - pos) >= mid) {
cow ++
pos = stall[i]
}
if (noc == cow) {
return true
}
}
False

**Example 1:**

```
Input: nums = [4,5,6,7,0,1,2], target = 0
Output: 4
```

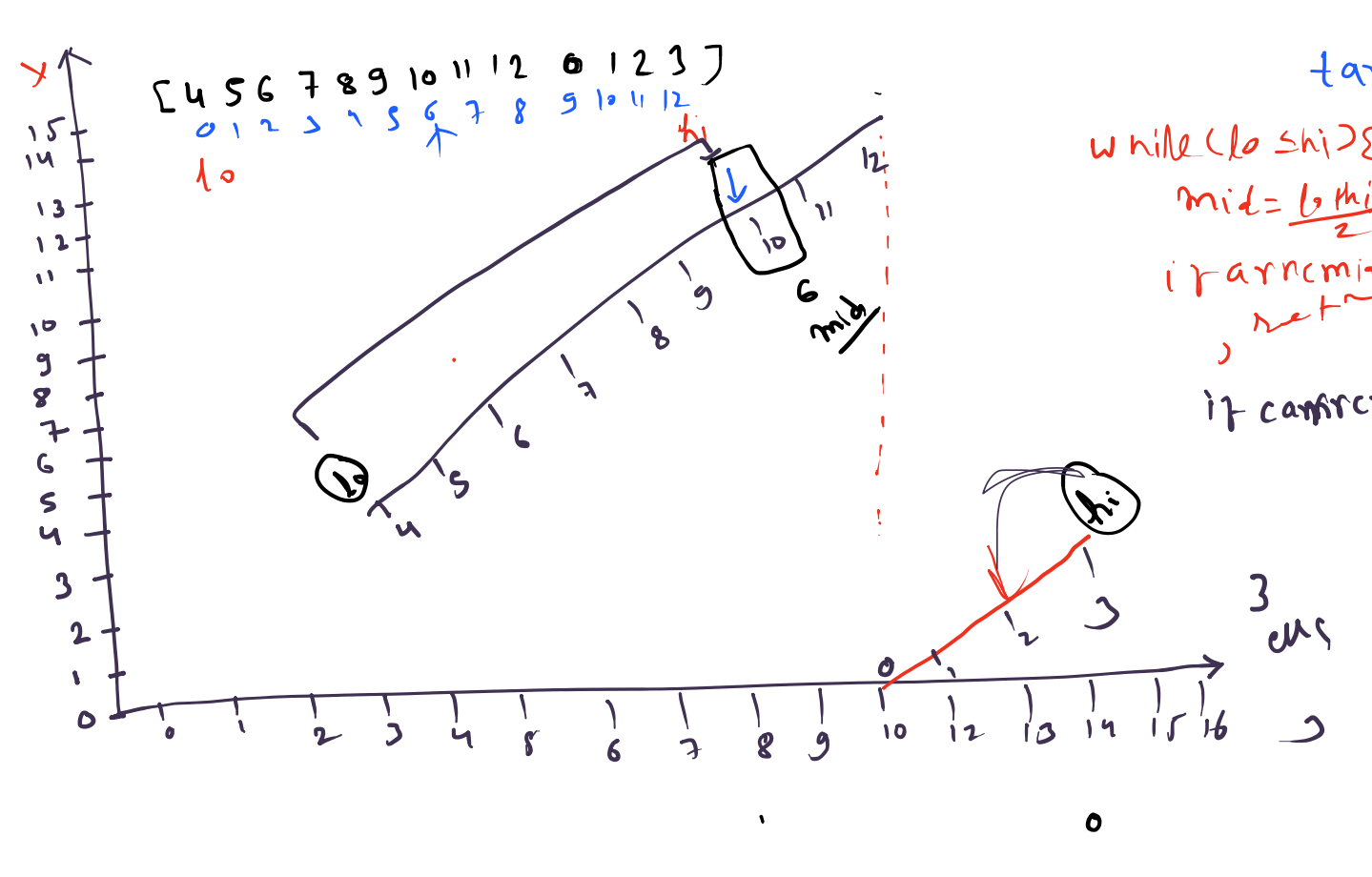
[4 5 6 7 0 1 2] X

**Example 2:**

```
Input: nums = [4,5,6,7,0,1,2], target = 3
Output: -1
```

**Example 3:**

```
Input: nums = [1], target = 0
Output: -1
```



target = 6

while (lo ≤ hi) {
mid = (lo+hi)/2
if arr(mid) == t) {
return
}
if (arr(mid) >= arr(lo)) {

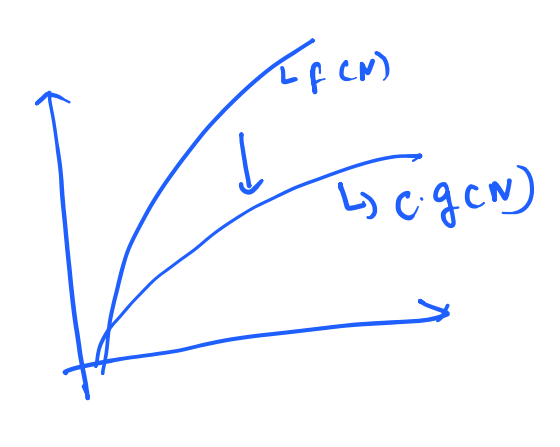
Time & Space Complexity

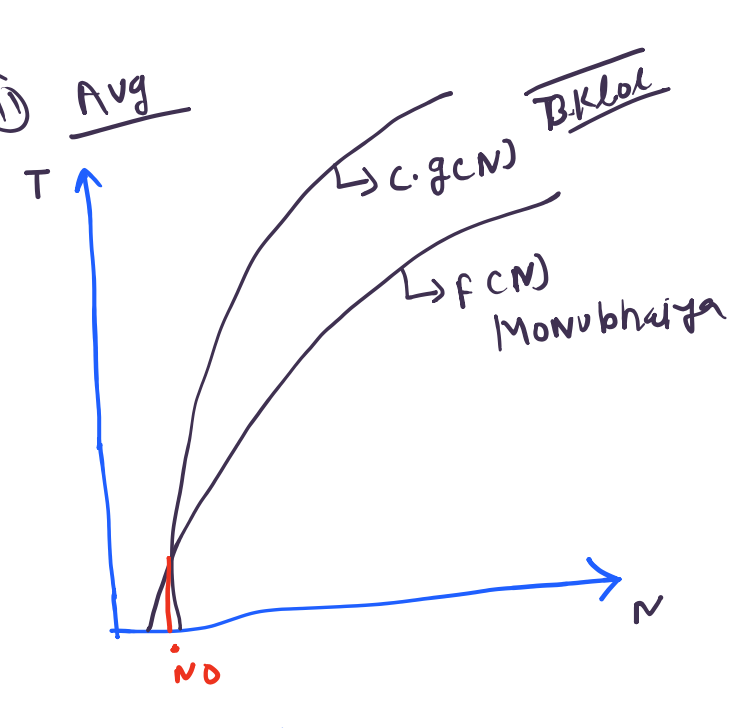
asymptotic notation vs Experimental Way

① Best ② Worst ③ Avg

$f(N) \leq c \cdot g(N)$ N > N0

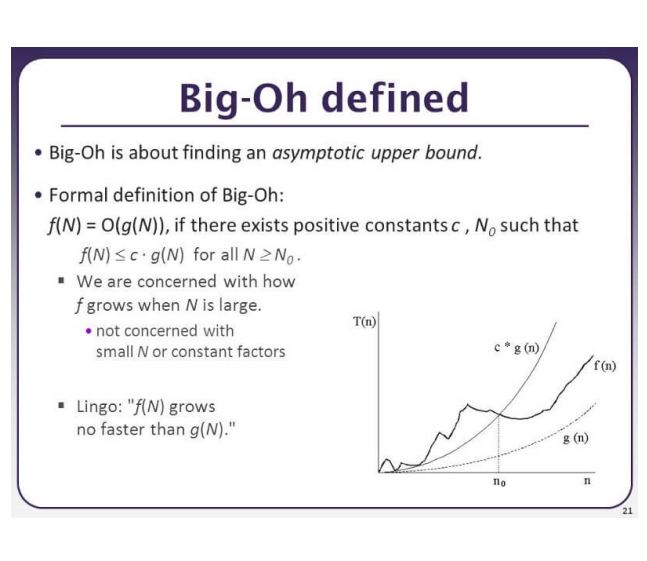
$f(N) \rightarrow O(g(N))$





$f(N) >= c g(n)$

$\rightarrow \Omega g(n)$

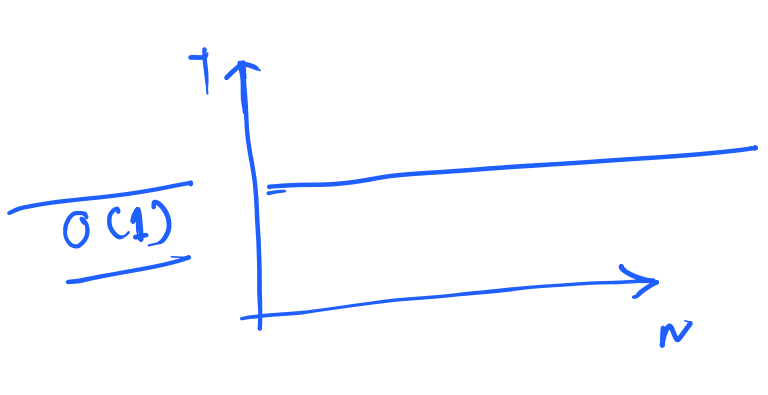


## Big-Oh defined

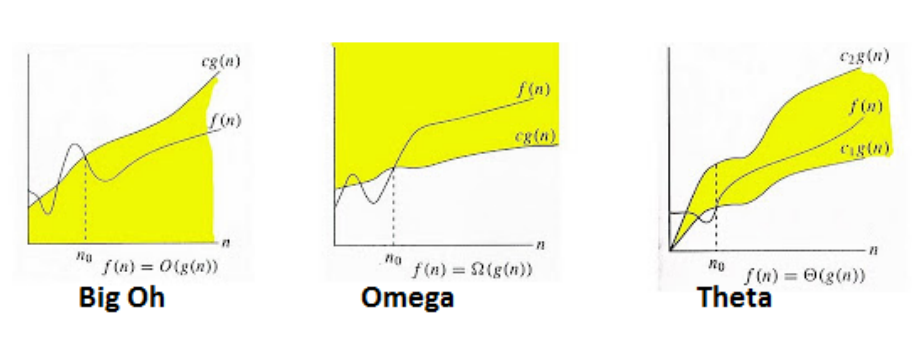
- Big-Oh is about finding an *asymptotic upper bound*.
- Formal definition of Big-Oh:
  $f(N) = O(g(N))$, if there exists positive constants $c$, $N_0$ such that $f(N) \leq c \cdot g(N)$ for all $N \geq N_0$.
  - We are concerned with how $f$ grows when $N$ is large.
    - not concerned with small $N$ or constant factors
  - Lingo: "$f(N)$ grows no faster than $g(N)$."

T(n)
c * g (n)
f (n)
g (n)
n0
n
21

Constant time



O(1)



$\boxed{T(n) = 2n^2 + 5n + 7} \rightarrow O(n^2)$

$(1000)^3$

$e^{1000}$

$T(n) = 2n^2 + e^n + e^n \log(n) + 7n^3 + 5$

```
public static int Search(int[] arr, int item) {
    for (int i = 0; i < arr.length; i++) {
        if(arr[i]==item) {
            return i;
        }
    }
    return -1;
}
```

i=0 i=1 i=2 i=3 i=4 ------ i=n-1

1 1 1 1 1 1

T(n) = n

T(n) = n+1 } O(n)

```
public static int Maximum(int []arr) {
    int max=arr[0];
    for (int i = 1; i < arr.length; i++) {
        if(arr[i]>max) {
            max=arr[i];
        }
    }
    return max;
}
```

→①

i=1 i=2 i=3 ------ i=n-1

1 1 1 1

T(n) = (n-1) + 2

= n+1

O(n)

```
public static int Search(int[] arr, int item) {
    int lo=0;
    int hi=arr.length-1;
    while(lo<=hi) {
        int mid=(lo+hi)/2;
        if(arr[mid]==item) {
            return mid;
        }
        else if(arr[mid]>item) {
            hi=mid-1;
        }
        else {
            lo=mid+1;
        }
    }
    return -1;
}
```

O(1)

T.C = O(k)

$N \rightarrow 1$

$\frac{N}{2} \rightarrow 1$

$\frac{N}{2^2} \rightarrow 1$

$\frac{N}{2^3} \rightarrow 1$

$N/4 \rightarrow 1$

$\frac{N}{2^K} \rightarrow 1$

$\frac{N}{2^K} = 1$

$2^K = N$

$\boxed{K = \log_2 N}$

```
while (i <= n) {
    System.out.println("Hey");
    i *= 2;
}
```

while (n>0) {
n = n/2
}

i=1 i=2 i=$2^2$ i=$2^3$ ---- i=$2^K$ => n

1 1 1 1 1

(K)

$2^K = n$

$K = \log(n)$

```
for (i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
        System.out.println("hey");
    }
}
```

→ n →n →1

independent

n×n×1 = $n^2$

i=1 i=2 i=3 i=4 ------- i=N

N N N N N

$n^2$

```
for (i = 1; i * i <= n; i++) {
    System.out.println("hey");
}
```

$i^2 \leq N$

$i \leq \sqrt{N}$

$\sqrt{N}$

```
for (i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j++) {
        for ( k = 1; k <= n; k++) {

        }
    }
}
```

i=1 i=2 i=3 i=4 ------- i=n

[N 2×N 3×N 4×N N×N]

[N + 2N + 3N + 4N + ----- N×N]

$N \times [\underbrace{1+2+3+4+ \cdots N}]$

$N\left[\frac{N \times (N+1)}{2}\right] \Rightarrow N^3$

```
for (i = 1; i <= n; i++) {
    for (int j = 1; j <= i * i; j++) {
        for ( k = 1; k <= n / 2; k++) {
            System.out.println("hey");
        }
    }
}
```

i=1 i=2 i=3 i=4 ------ i=N

$\frac{N}{2}$ $2^2 \times \frac{N}{2}$ $3^2 \times \frac{N}{2}$ $4^2 \times \frac{N}{2}$ $N^2 \times \frac{N}{2}$

$\frac{N}{2}[\underbrace{1+2^2+3^2+4^2+ \cdots + n^2}]$

$\frac{N}{2}\left[\frac{n(n+1)(2n+1)}{6}\right] = N^4$

```
for (i = n / 2; i <= n; i++) {
    for (int j = 1; j <= n / 2; j++) {
        for ( k = 1; k <= n; k = k * 2) {
            System.out.println("hey");
        }
    }
}
```

$\frac{n}{2}$ $\frac{n}{2}$ log(n)

$n^2 \log(n)$

```
for (i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j += i) {
        System.out.println("hey");
    }
}
```

i=1 i=2 i=3 i=4 ------ i=N

N $\frac{N}{2}$ $\frac{N}{3}$ $\frac{N}{4}$ $\frac{N}{N}$

$N[1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} \cdots \frac{1}{N}]$

N×log(N)

```
public static void Sort(int[] arr) {
    // TODO Auto-generated method stub
    for (int turn = 1; turn < arr.length; turn++) {
        for (int i = 0; i < arr.length - turn; i++) {
            if (arr[i] > arr[i + 1]) {
                int temp = arr[i];
                arr[i] = arr[i + 1];
                arr[i + 1] = temp;
            }
        }
    }
}
```

(log n) × log(n)

$(\log n)^2$

turn=1 turn=2 turn=3 turn=4

4 3 2 1

$\frac{n \times (n-1)}{2} = n^2$

```
public static int SubarraySum(int[] arr) {
    int ans = Integer.MIN_VALUE;
    for (int i = 0; i < arr.length; i++) {
        int sum = 0;
        for (int j = i; j < arr.length; j++) {
            sum += arr[j];
            ans = Math.max(ans, sum);
        }
    }
    return ans;
}
```

→ i=0

i=1 i=2 --- i=n-1

N N-1 N-2 1

$\simeq n^3$

$n^2$

$n <= 10^5$

$(10^5)^3$

$10^{15} >= 10^8$